Data Mining Techniques Assignment

Nicholas Boidi, Pablos Tselioudis Garmendia, and Filippo Donghi

Vrije Universiteit Amsterdam, Amsterdam, Netherlands
Group Number: 65
Student Numbers: 2892127, 2891511, 2888236

1 Introduction

This report describes our solution for Assignment 2 of Data Mining Techniques. The task is to rank hotel properties per search query (`srch_id`) so that booked hotels appear at the top, followed by clicked hotels, measured with NDCG@5 on the Kaggle competition server.

The data contains approximately five million rows for training and five million for testing. Each row is one displayed property for one search query, with rich contextual, property, and competitor information. We followed the data mining workflow suggested by the assignment, moving from related work and business framing into exploratory analysis, then feature engineering and the split strategy, then modeling and evaluation with at least two methods, and finally deployment considerations including ethical AI bias analysis and mitigation.

Our main model is a LightGBM LambdaMART ranker. It improves strongly over a recommender-style popularity baseline, and a reweighted variant reduces the performance gap between family and non-family queries.

2 Related Work and Business Understanding

The business objective is to improve ranking quality in hotel search, because better ranking increases user satisfaction and conversion (clicks/bookings). This is a recommender systems problem with context-aware ranking constraints.

Prior recommender literature highlights three relevant ideas. First, collaborative and latent-factor methods are effective at capturing user-item preference patterns [1]. Second, pairwise and listwise ranking methods (e.g., LambdaMART) are highly suitable when the evaluation target is NDCG [2]. Third, implicit-feedback recommenders (click/view/book interactions) require careful weighting of signal strength [3].

Work from the original Expedia personalized hotel search challenge also emphasizes the value of hotel characteristics, location attractiveness, user purchase history, and competitor information, and reports combining multiple ranking models for the final hotel ordering [4]. This motivated our two-model setup:

- A recommender-style popularity model with Bayesian smoothing (strong interpretable baseline).
- A feature-rich LightGBM LambdaMART model as our final method.

Table 1. Training/validation statistics from our reproducible run.

Statistic	Value
Training rows	3,966,833
Validation rows	991,514
Training queries (`srch_id`)	159,836
Validation queries (`srch_id`)	39,959
Booking rate	2.79%
Click rate	4.47%

3 Data Understanding

3.1 Dataset Profile

We used the provided competition data only. We loaded the full training set of 4,958,347 rows and split by `srch_id` (query-level) into 80% training and 20% validation. The resulting split statistics are shown in Table 1.

The low booking rate confirms heavy class imbalance. Since NDCG@5 is query-wise and top-heavy, calibration near the top ranks is much more important than global classification accuracy.

3.2 Exploratory Findings

We performed EDA on missingness, target sparsity, and key relationships:

- Competitor fields (`comp`) and historical visitor fields have substantial missingness, as shown in Figure 1.
- **Position bias:** when results use Expedia's own sort (`random_bool=0`), hotels near the top are booked far more often simply because of where they sit on the page. Under random sort (`random_bool=1`) the booking rate stays almost flat across positions (Figure 2). This finding is what led us to compute the popularity priors from random-sort rows only.
- Price has a non-linear relationship with relevance. Very low and very high prices can both underperform depending on context, and raw price spans seven orders of magnitude because of mixed country conventions.
- Relative features inside a search list (e.g., price rank within `srch_id`) are more informative than absolute values alone.

4 Data Preparation

4.1 Target Construction

Following the competition definition, we used graded relevance:

$$\text{relevance} = 5 \cdot \texttt{booking\_bool} + 1 \cdot (\texttt{click\_bool} \wedge \neg \texttt{booking\_bool}).$$

This keeps booked items as the highest-signal outcomes while still rewarding clicks.

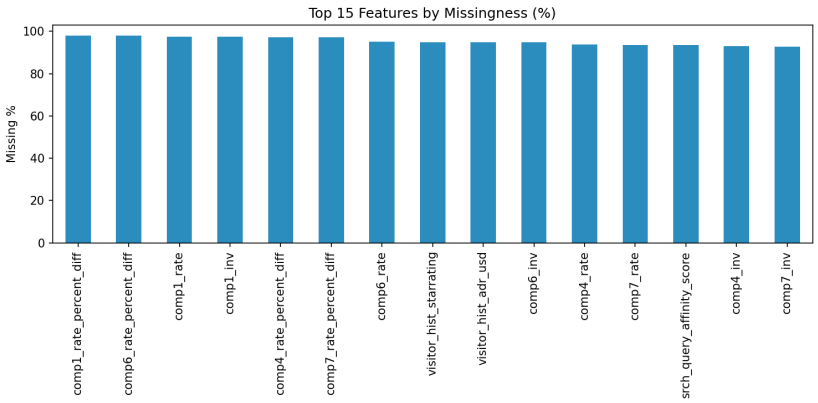

Fig. 1. Top-15 features by percentage of missing values.

4.2 Train/Validation Strategy

To avoid leakage, we split by `srch_id` (query-level split), not by row. This prevents rows from the same search appearing in both train and validation.

4.3 Feature Engineering

We engineered six groups of features:

- **Raw context/property features:** site, destination, stars, review score, promotion, booking window, party size, etc.
- **Price cleaning:** `price_usd` is clipped at $10,000 and log-transformed, and the per-night price (`price_usd` / `srch_length_of_stay`) is transformed the same way. Raw prices span seven orders of magnitude because of mixed country conventions such as per-night versus whole-stay pricing and prices shown with or without tax, which makes log-scaling necessary.
- **Competitor signals:** counts of competitors where Expedia is cheaper, pricier, or unavailable, plus the maximum absolute price difference. These replace naïve column means over the mostly-missing competitor fields.
- **Within-query relative features:** z-scores of log-price, location score, star rating, review score, and log-historical price within each `srch_id`. We also add rank percentiles of price, location score, star rating, and review score, along with the query length, meaning the number of candidate hotels per search.
- **Popularity priors** (fit on the training fold only): Bayesian-smoothed property click-through and booking rates, mean historical position, destination-property relevance, destination-star affinity, and overall destination booking rate.
- **Visitor-history and temporal features:** differences from historical star and price preferences, total party size, per-person per-night price, month, and day-of-week.

Fig. 2. Mean booking rate by display position. Under Expedia's normal sort, hotels in top positions are booked far more often. Under random sort the rate stays nearly flat. The gap between the two curves reflects position bias rather than hotel quality, which is why the popularity priors use `random_bool=1` rows only.

For missing values, we relied on LightGBM's native NaN handling for raw features and used smoothed fallback means for target-encoded priors.

5 Modeling and Evaluation

5.1 Metric

We evaluated with NDCG@5 per query and averaged over queries, matching the competition objective.

5.2 Technique 1: Recommender-Style Popularity Baseline

As a recommender systems technique, we ranked hotels using historical popularity priors with Bayesian smoothing:

- Global property signal: smoothed click-through rate per `prop_id` (`prop_ctr`).
- Destination-conditional signal: smoothed relevance for destination-property pairs.
- Final score: equal-weighted average of the two priors, 0.5 each.

This baseline is simple, scalable, and interpretable, and it captures collaborative signal from aggregate user interactions.

Table 2. Validation NDCG@5 results.

Model	NDCG@5
Popularity recommender baseline	0.2417
LightGBM LambdaMART (no mitigation)	0.3895
LightGBM LambdaMART (bias-mitigated)	0.3894

5.3 Technique 2: LightGBM LambdaMART

Our main model is `LGBMRanker` with the LambdaMART objective, a listwise learning-to-rank algorithm that directly optimises NDCG [2]. The model receives a *group* parameter specifying the number of rows per query, enabling it to compare candidates within the same search. Key parameters:

- `learning_rate=0.05`, `num_leaves=255`
- `feature_fraction=0.85`, `bagging_fraction=0.85`, `bagging_freq=5`
- `min_child_samples=100`
- `label_gain=[0,1,0,0,0,31]` (maps relevance 0/1/5 to gains $2^r - 1$)
- Early stopping on validation NDCG@5 with patience 100, with the best iteration found at 284

5.4 Results

Table 2 shows local validation performance.

The LambdaMART model improves over the recommender baseline by 0.1478 absolute NDCG@5 (+61%), confirming that a dedicated ranking objective with query-relative and unbiased target-encoded features provides substantial additional signal over popularity alone. Figure 3 shows the top-20 features by LightGBM gain importance.

5.5 What Did Not Work Well

Four observations from our iteration cycle:

- A pointwise regression objective (MSE on graded labels) ranked poorly compared to LambdaMART because it does not directly optimise the list-level NDCG criterion.
- Within-query relative features such as z-scores and rank percentiles sit near the top of the gain-importance plot. Earlier iterations that leaned more on raw absolute features ranked worse at the top, which matches the recommender literature on the value of relative signals within a list.
- Plain column-wise means over the mostly-missing competitor fields (94–98% missing) would mostly average noise, so we encoded them as counts of competitors where Expedia is cheaper, pricier, or sold out. The count form is naturally immune to the missing values, because a missing comparison simply does not add to any count.

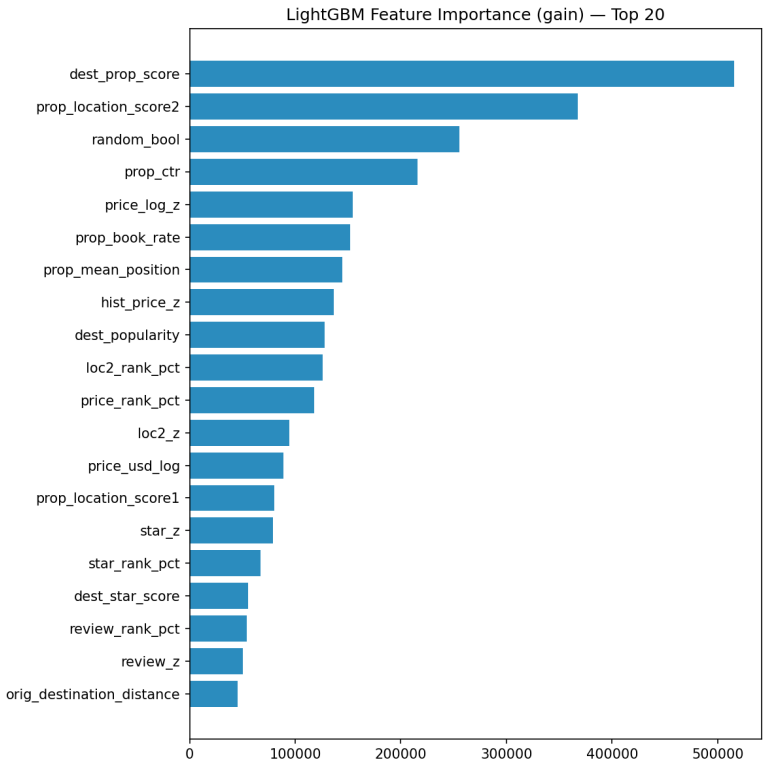

Fig. 3. Top-20 features by LightGBM gain importance. Target-encoded priors and within-query relative price features dominate.

– Feeding hotel and destination identifiers to LightGBM as native categorical features lowered validation NDCG@5. Their high cardinality let the trees memorise individual hotels and overfit the training fold, so the identifiers were kept only as inputs to target encoding.

6 Deployment and Ethical AI

6.1 Scalable Deployment Discussion

A production deployment can be implemented as a two-stage architecture:

– **Offline batch layer:** refresh popularity priors and retrain LambdaMART model daily/weekly.
– **Online ranking layer:** compute lightweight query-relative features in real time, score candidate hotels, and return top-k list.
– **Monitoring:** track NDCG proxies, CTR/booking rates, latency, drift, and fairness gaps over time.

The feature design in this report fits this architecture well, since heavy aggregates are precomputed offline while per-query transforms stay low-latency.

6.2 Bias Detection

We audited group fairness across two user-query segments:

- **Family queries:** `srch_children_count > 0`
- **Non-family queries:** `srch_children_count = 0`

Using NDCG@5 by group as the fairness metric:

- Before mitigation: family 0.4063, non-family 0.3841, gap 0.0222.

Family queries score higher on average, likely because families search more specifically (clearer destination and party-size constraints narrow the relevant set). A positive gap still represents unequal treatment across user segments.

6.3 Bias Mitigation and Effectiveness

We applied a **pre-processing reweighting** technique. During model training, rows from family queries received weight 1.6 while other rows kept weight 1.0. This tests whether giving the smaller query segment more influence can reduce group-level ranking disparity.

After mitigation:

- Family NDCG@5: 0.4040
- Non-family NDCG@5: 0.3847
- Gap: 0.0193

The mitigation raised non-family NDCG@5 from 0.3841 to 0.3847 while family NDCG@5 decreased from 0.4063 to 0.4040. Together this narrowed the gap from 0.0222 to 0.0193, a reduction of about 13 percent, with only a very small global NDCG@5 decrease from 0.3895 to 0.3894. The remaining gap is partly intrinsic: family queries are easier to rank because a clear destination and party-size constraints narrow the set of relevant hotels. Closing it further would need a more targeted method such as group-specific calibration or adversarial debiasing.

7 What We Learned

This assignment improved our understanding in three ways:

- **Ranking mindset:** optimizing query-level top-k quality is different from standard classification.
- **Feature engineering impact:** relative within-query signals were more valuable than many raw features.
- **Responsible modeling:** fairness auditing and mitigation can be integrated with small performance cost.

Main challenges were data scale, missingness, and balancing relevance optimization with fairness. Unexpectedly, a simple popularity recommender already gave a strong baseline, showing the value of historical interaction priors in travel recommendation.

8 Conclusion

We developed and evaluated two ranking approaches for hotel recommendation, a recommender-style popularity baseline at a local validation NDCG@5 of 0.2417 and a LightGBM LambdaMART model with engineered contextual features at a local validation NDCG@5 of 0.3895. The LambdaMART model benefits from a dedicated ranking objective, price cleaning, unbiased target-encoded popularity priors, within-query relative features, and destination-level collaborative signals. The resulting pipeline is reproducible, scalable to production-style ranking, and aligned with both performance and ethical AI requirements of the assignment.

References

1. Koren, Y., Bell, R., Volinsky, C.: Matrix factorization techniques for recommender systems. Computer **42**(8), 30–37 (2009)
2. Burges, C.J.C.: From RankNet to LambdaRank to LambdaMART: An overview. Microsoft Research Technical Report, MSR-TR-2010-82 (2010)
3. Rendle, S., Freudenthaler, C., Gantner, Z., Schmidt-Thieme, L.: BPR: Bayesian personalized ranking from implicit feedback. In: UAI 2009, pp. 452–461 (2009)
4. Liu, X., Xu, B., Zhang, Y., Yan, Q., Pang, L., Li, Q., Sun, H., Wang, B.: Combination of diverse ranking models for personalized Expedia hotel searches. arXiv:1311.7679 (2013). `https://doi.org/10.48550/arXiv.1311.7679`
5. Jannach, D., Zanker, M., Felfernig, A., Friedrich, G.: Recommender Systems: An Introduction. Cambridge University Press, Cambridge (2010)
6. Wikipedia contributors: Discounted cumulative gain. `https://en.wikipedia.org/wiki/Discounted_cumulative_gain` (accessed 2026-04-23)